

File-Integrity and Merkle-Trees

Due Dates/Milestones:

- Milestone 1 Check-In: 14/05/2026
- Milestone 2 Check-In: 28/05/2026
- Expected Completion: 11/06/2026

You are tasked with constructing a file integrity library that would be used for files and validating the integrity against a set of hashes that provided in a separate format. There are three phases to this component:

1. MetaData Interpretation (Part 1):

You will need your library to read and interpret the `tpk` file that will contain an identifier, filename, size, hash list and chunk list.

- Loading and interpreting a package file.
- Opening the `.tpk` file and reading the contents of the file.
- Loading a file that is referenced within the package file.
- Retrieve the identifier, filename, filesize, number of hashes, hashes, number of chunks, and chunk tuples.
- Construct a suitable object and validate the number of hashes and chunks with their actual entries.

2. File Integrity Validation with Chunk-List (Part 2):

Once the `tpk` file has been loaded and interpreted correctly, you will need to reference the file that the `tpk` has included. This will be required for you to proceed to verify each chunk of the file that is referenced inside the `tpk` file.

- Check if the file exists.
 - If the file does not exist, a file should be created in its place.
 - If the file exists but the size does not match, it should be recreated and the size of the file should be expanded.
- Check if the file is complete or incomplete.
- Be able to validate which blocks are complete and not complete.

3. File Integrity Validation with Merkle-Tree (Part 3):

- Construct a merkle-tree from the MetaData.
- Compute Hashes of a subtree, use the “real chunk” hash and compare to the one listed
- Compute non-leaf hashes from their child nodes and compare it against the hash listing from the metadata/merkle tree
- Compute the minimum set of hashes needed to represent the completeness or incompleteness of the file
- Write to a subset of data using a root of a subtree’s hash.

Part 1 - Loading the Package File

The package file format is a text format that will need to be parsed by your program. The package file format has the following fields.

- **ident**, **string** (1024 characters max), the identifier is used within the network to identify the same packages.
- **filename**, **string** (256 characters max), This is used to help save and locate the file to update when data is sent to it.
- **size**, **byte**, specifies the size in bytes
- **nhashes**, **byte**, specifies the number of hashes that are pre-computed from the original file. There must be only 2^{d-1} hashes which will correspond to the hashes of all non-leaf node
- **hashes**, **string**[$2^{(d-1)} - 1$] (64 characters for each string), these correspond to the number hashes in the previous **nhashes** field.
- **nchunks**, **byte**, specifies the number of chunks. The number of chunks must be a 2^d value.
- **chunks**, **struct**[$2^{(d-1)}$], each chunk have the fields: hash, offset and size.
 - **hash** refers to a string (64 characters), corresponding to the datablock hash value
 - **offset**, **byte**, is the offset within the file
 - **size**, **byte**, is the size of the chunk in bytes

The format below gives an outline to the structure of a **.tpk** file. Refer to the **resources** folder in the scaffold for a real example.

```
ident: <identifier>
filename: <filename>
size: <size in bytes>
nhashes: <number of hashes that are non-leaf nodes>
hashes:
    "hash value"
    ...
nchunks: <number of chunks, these are all leaf nodes>
chunks:
    "hash value",offset,size
    ...
```

Part 2 - File Verification - Chunk Chain Validation

The file validation method within this component is to identify what chunks are considered completed and what is not considered completed. Given the `.tpk` file, it will refer to a file that may exist.

- If it exist, the size of the file will need to be checked.
- If it does not exist, a file of the exact size referred by the metadata will need to be constructed.

In addition, this component will need to compute the hashes of data blocks and check it against the expected hash values within the metadata file. The verification will use the hashing algorithm SHA256 which is a built-in hashing algorithm in C#.

Your program will need to implement the following.

- Read datablocks within the file itself
- Compute hashes from the datablocks themselves
- Report the set of datablocks which are complete and incomplete
- Extract individual datablocks via an index.
- Report the completed status of a datablock itself.

Part 3 - File Verification - Merkle-Tree Validation

As another layer of verification which will mitigate against collisions, reduce data requirements and be usable when hash set provided in the `.tpk` file is minimised is to construct a merkle-tree.

The merkle-tree construction will need to be able to retrieve subtrees within the merkle-tree. These would correspond to the root node's left and right node and other descendants. The expected hashes encoded within the metadata component are used to check against when computed.

Your merkle-tree implementation should include the following.

- Constructing a merkle-tree from a relevant file object
- Retrieving subtrees within the merkle-tree
- Retrieving all expected hashes associated with the metadata
- Computing the minimum-set of hashes that represent the completion state of the file.
- Write data based on a subtree-range using the hash of a non-leaf node within the merkle-tree itself.

How does a MerkleTree work?

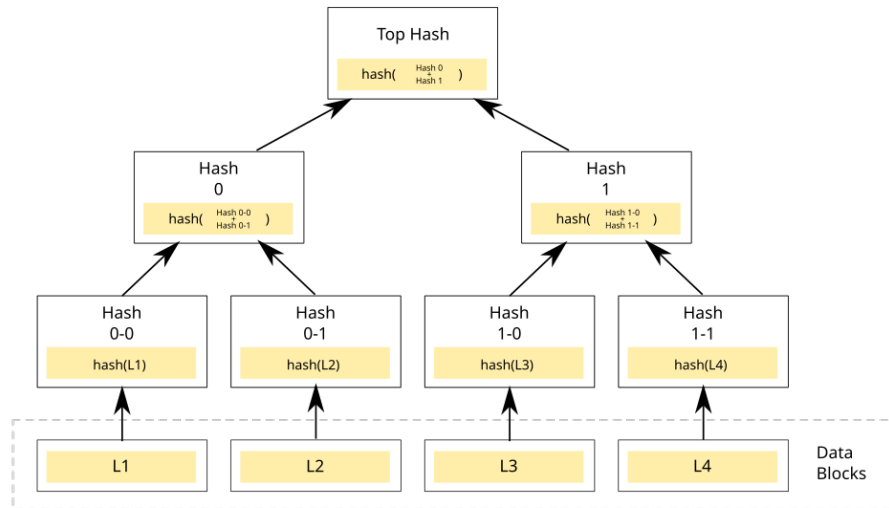


Figure 1: Merkle Tree/Hash Tree - Wikipedia

The following is in relation to the .tpk file and your merkle tree's construction.

A merkle tree has two hashing operations.

1. Hashes that operate on chunks/datablocks - These are computed from the data in the file itself.
2. Hashes that operate on a hash-pair - A hash-pair is when you have two chunks or two non-leaf nodes and you concatenate the hashes to then compute a new hash which then is assigned to its parent.

Note: The chunks or nodes selected must be siblings.

3. Operation describe in **2.**, occurs until we have computed the root's hash value.

Representing the completion state To represent the completion state of a file using a merkle-tree, you will need to represent it using the non-leaf nodes of the tree itself. This is by finding the roots of subtrees within the tree itself.

Use the following as a guide with a tree of 4 chunks (and 7 nodes in total).

- If the file is complete, therefore all chunks match what is outlined in the chunk list, this can be represented as hash that the root of the tree has.

- If the first two chunks of the file are completed, the root's left child hash would be returned.
- If only the first and last chunks of the file are completed, the first and last chunk hashes would be returned.
- If the last half of the file completed (third and last chunk) and the first chunk completed, the root's right child's hash and the hash of the first chunk.
- If the first half of the file is completed and the last chunk is completed, the hash of the root's left child and hash of the last chunk is returned.

Submission, Milestones, Testing, and Instructor Test Cases

Instructor test cases and files have been included within the project itself but test cases will be rolled out with the cut-off being 3 days before the submission of Milestone 2.

You will have 3 submission portals which include Milestone 1 and Milestone 2.

You are encouraged to write your own test-cases and attempt to cover branches and cases not covered by the instructor's test cases.

As a hint, the instructor test cases are likely to provide typical-scenarios/happy scenarios and not cases where bad-input is at play. Consider cases which may not be well specified or may require you make assumptions. You can discuss your assumptions with the instructor to check for their validity.

Checklist

Use this as a checklist to ensure you have completed the assessment.

- Implement methods and classes related to **Metadata/.tpk**
- Implement methods and classes related to **FileFormat**
- Implement methods and classes related to the **MerkleTree**
- Implement test cases as part of your own implementation
- Implementation passes the instructor test cases
- Write down assumptions and make notes regarding your solution.

Instructor Comments and Potential “Gotchas”

This is your first data-structures related assessment and can represent another significant hurdle to overcome. After building a Key-Value database. Building a merkle-tree is a significant hurdle because it is a variant of a standard data structure with very practical use cases within file transfers and networking.

I would encourage everyone to complete the binary-tree and binary-heap homework before attempting this assessment. It is also worth getting familiar with file-reading and writing. In addition, use the utilities provided to help build your solution.

- Use tools like **xxd** or some other kind of a hex-viewer when you need to check the contents of a file. Alternatively, you can **Read** the data using **BinaryReader** and retrieve an array of the data segments themselves.
- The files you are working on do not have to be big to test it with, 4096 bytes is enough to split it into 4x 1024 chunks. For setting up your own tests, you can do the following on the next page.
- Ask questions early and often, create experiments and give it your best attempt, you learn a lot more by just trying and getting feedback.

Creating your own test files

Linux/WSL Method

1. How could I create my own file with garbage data? (For Part 2)

To do this using the command line you can run `cat /dev/urandom | head -n N > filename`, where **N** is the number of bytes you want to read and **filename** is the name you want for the file.

2. How could I split the file into chunks? (For Part 2)

You can use the command `split -n K` where **K** is the number of chunks you want to generate. For example, you can run `split -n 4 filename`, this will generate 4 chunks of the file you had created prior. These files are prefixed with **x****, for 4 chunks, you will get the following files. **xaa**, **xab**, **xac**, **xad**.

3. Use `sha256sum` on each chunk to build the hashes for the chunks. (For Part 2)

A simple way to do this is to simply run `sha256sum x*` which will run the command on all files that start with **x**. You will get the following output.

```
5c60ab506c99acb92d5f215ed0ec398393c99cfbcb12dc21e1f1965885563c08 xaa
ad6a6b45534148001192599c4634f9150fd7024287255cdad559755093fb0335 xab
d619fcf6f80f81dca88829a73a24a973c4b879ad8a35ffd06251b1048cab404d xac
c66cdce421390f74d085dc40ecac9fd7ac653ecc5c6ab5a0d7154cacfcd0c9b5 xad
```

Do note, the hashes listed above will not match the ones in this file (or it is very unlikely for them to).

Also note, if you have more than 1 file that has a hash that matches, this **may** cause issues here! Double check that your data is randomised or provide the input yourself to ensure it is to create a different hash for each chunk. Validate this with `xxd`.

4. Computing intermediary hashes - With **K** chunks, you have **K-1** non-leaf hashes. (Part 3)

In a binary tree where there are 4 leaf nodes, you will have 3 non-leaf nodes. To now compute the intermediary hashes, you will need to use the hash from chunk 0 and 1 to produce the parent hash.

To do this using the shell, you can do the following, (Concatenate the hash of chunk 0 and 1) and run it through `sha256sum`. Make sure you do not include a newline (This is why `echo` is using the `-n` flag).

```
echo -n "5c60ab506c99acb92d5f215ed0ec398393c99cfbcb..." | sha256sum
```

You will notice that it produces:

```
89a3edf5f43e379ebdce3c45d218f1a20ccd46b2968d1aec66c59a1af2a6c1ae.
```

Since this is a tree with 3 levels, chunks 0 and 1 parent is the root's left child.

Do the same for chunk 2 and 3 which will result in the root's right child.

After you have both left and right, you can then take the hashes from both and perform the same operation to produce the root's hash.

5. You should have all the data now, you can then generate a random identifier for a **tpk** file and enter it in as the format has specified.

Windows Only Method - C# Guide

1. How could I create my own file with garbage data? - **GenRandomSlice(int N)** (Part 2)

We won't be creating a file first, we will need to build a way to create random data/noise.

Create a program that will have a method that can create a **List<byte>** of size **N**.

To create a file of a specific size with **random** data, consider creating a simple program that can write to a file **N** bytes. Consider the file size to be a power of 2 for simplicity such as 16, 32, 64, 128... etc.

2. Once you have a method that can create a **List<byte>** of size **N**, you can then have a method that can create **K** Chunks. - **MakeChunksWithRandomSlices(int K)** (Part 2)

Make another method that will use your previously constructed. If you specify **K** chunks, this should result in **K** calls to **GenRandomSlice** of which you should have a **List<List<byte>>** at the end of it all. This allows you operate on individual slices for hashing and write to a file in sequence.

3. Retrieving the hashes from a byte slice - Use **SHA256** class and the **ComputeHash** method. (Part 2)

You will need to call **SHA256.Create()** static method to create the object and then call **ComputeHash** with a **byte[]** array. For each chunk, meaning, each **List<byte>**, convert the **List<byte>** to a **byte[]** (check for available methods) and pass it into **ComputeHash**. This will create the hash from the slice.

4. Computing intermediate hashes, with **K** chunks, you will have **K-1** non-leaf hashes. - **ComputeParentHash(? left, ? right)**(Part 3)

Note: The **?** are specified there because it is up to you to specify the types here. Consider Polymorphism here!

In a binary tree where there are 4 leaf nodes, you will have 3 non-leaf nodes. To now compute the intermediary hashes, you will need to use the hash from chunk 0 and 1 to produce the parent hash.

Firstly, you will need to generate a **Hex** string, you can do this by doing the following on a **byte**.

```
byte b = 16;  
string hexString = b.ToString("x2"); // Will be 10 since hex is base 16
```

Concatenate both the left and right object's hex strings and compute the hash from it. You will need to convert the string to bytes again which you can do via **Encoding** class.

Once you have used **ComputeHash** on that string, you have the parent's hash. You can then do it to the next pair.

You can repeat the process for the non-leaf nodes and compute the hash of the root as well. Use the given testfiles as a way to check if you have resolved it.

5. You should have all the data now, you can then generate a random identifier for a **tpk** file and enter it in as the format has specified.

Since you have constructed a **C#** program, you could construct a **.tpk** file at this point.

Potential Gotchas

- It can be difficult to figure out where to start. Fortunately, the assessment has been planned to build up from metadata to the merkle-tree. It is advisable to solve the assessment in the order outlined.
- Within the metadata part, parsing the file given is going to be the main challenge. There is a structure to this file and you will need to ensure the structure can be read predictably by your implementation. Set up some simple test cases that allow you test the fields you are extracting.
- Within the **.tpk** file, you should not confused the expected hashes listed there with the computed hash of the chunk itself.
- Within the first file-verification part, there are going to be three significant challenges.
 1. Reading the chunk data and not making off-by-1 errors (very common, especially with files that are not split nicely)
 2. Using the SHA256 algorithm is going to be a new, while it can seem scary, it isn't that bad, please refer to SHA256 Class from Microsoft Developer Network.
 3. Retrieving blocks precisely and computing the completeness of the file itself can be a bit of a challenge but once 1 and 2 are complete, 3 should be within reach

- Within the second file-verification part, the tree and the nodes may have some misconceptions or one may experience difficulties with construction.
 1. Always remember that a MerkleTree is a BinaryTree
 2. It is also a perfect binary-tree in this case
 3. It may be appropriate to use inheritance to describe the different types of nodes you have in the tree. Specifically, if one is a leaf and the other is a non-leaf.
 4. Extracting the minimum-set of hashes should be left last, this is the main-perk of the data structure but ensure you have the other parts working correctly and you can extract the hash from chunks. Set up test cases that compute the hash of a non-leaf node from the children of two completed leaf nodes.
- This is a task that requires asking questions and interrogating the contents of it. Do not be afraid to ask questions during the Q&A session or via Teams message.

Good Luck and Happy Coding! -

Version: 2026-04-30,Release-1